## Merge Sort

Merge Sort is a divide-and-conquer sorting algorithm. It divides an array into halves until each part has one element, then merges sorted parts.

Divide: split the range into two halves.

Conquer: recursively sort both halves. A one-element range is already sorted.

Combine: merge the two sorted halves by repeatedly taking the smaller front element.

```
void merge(vector<int>& a, int l, int m, int r) {
    vector<int> temp;
    int i = l, j = m + 1;
    while (i <= m && j <= r) {
        if (a[i] <= a[j]) temp.push_back(a[i++]);
        else temp.push_back(a[j++]);
    }
    while (i <= m) temp.push_back(a[i++]);
    while (j <= r) temp.push_back(a[j++]);
    for (int k = 0; k < (int)temp.size(); k++)
        a[l + k] = temp[k];
}

void mergeSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}
```

| Property | Merge Sort |
|---|---|
| Best / average / worst time | O(n log n) |
| Extra space | O(n) for temporary array |
| Stable? | Yes, if equal values are taken from the left half first (<=). |
| In-place? | No, not in this standard implementation. |

## Quick Sort

Quick Sort chooses a pivot, partitions the array so values ≤ pivot go to its left and larger values go to its right, then recursively sorts the two sides.

The pivot is placed in its final sorted position after partition.

The recursive calls do not include the pivot again.

This example uses the last element as pivot.

```
int partitionArray(vector<int>& a, int low, int high) {
    int pivot = a[high], i = low - 1;
    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
            swap(a[i], a[j]);
        }
    }
    swap(a[i + 1], a[high]);
    return i + 1;
}

void quickSort(vector<int>& a, int low, int high) {
    if (low >= high) return;
    int p = partitionArray(a, low, high);
    quickSort(a, low, p - 1);
    quickSort(a, p + 1, high);
}
```

- **pivot = arr[high]:** The last element of the array (or subarray) is chosen as the pivot.
- **i = low - 1:** The pointer i is initialized to one position before the start of the array (or subarray).
- **for j from low to high - 1 do:** Iterate through each element in the array (or subarray) from low to high - 1.
- **if arr[j] <= pivot then:** If the current element arr[j] is less than or equal to the pivot, increment the pointer i and swap arr[i] with arr[j].
- **swap arr[i] with arr[j]:** Swap the current element arr[j] with the element at the ith position, ensuring that smaller elements are moved to the left.
- **swap arr[i + 1] with arr[high]:** After the loop, swap the pivot element with the element at i + 1 to place the pivot in its correct position.
- **return i + 1:** The function returns the index of the pivot, which is now in its correct sorted position.

| Property | Quick Sort |
|---|---|
| Best / average time | O(n log n) |
| Worst time | $O(n^2)$, when partitions are repeatedly very unbalanced |
| Auxiliary stack | Average O(log n), worst O(n) |
| Stable? | Usually no |

| | |
|---|---|
| In-place? | Usually yes, apart from recursion stack |

## Backtracking

Backtracking builds a solution one decision at a time. If a partial solution cannot lead to a valid answer, undo the last decision and try another.

```
void solve(State state) {
    if (complete(state)) {
        record(state);
        return;
    }
    for (each choice) {
        if (valid(choice)) {
            make(choice);
            solve(updated_state);
            undo(choice);
        }
    }
}
```

State: what have we chosen so far?

Choices: what can we try next?

Constraint: is the choice allowed?

Base case: is the solution complete?

Undo: restore the previous state before trying another branch.